

Programming Languages and Environments (Lecture 21)

LEI - Licenciatura em Engenharia Informática

João Costa Seco (joao.seco@fct.unl.pt)



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Alonzo Church (1903 - 1995)

- Foundational work on Programming Languages.
- Devised the formal system of Lambda Calculus:

$$E ::= x \mid \lambda x.E \mid E E$$

$$(\lambda x.E) E' \longrightarrow E\{E'/x\} \qquad \frac{E \longrightarrow E''}{E E' \longrightarrow E'' E'}$$

Recall Lecture #1



Lambda Calculus - Call by name



```
type lambda =  
  | Num of int  
  | Add of lambda * lambda  
  
  | Var of string  
  | App of lambda * lambda  
  | Fun of string * lambda  
  
let f = Fun ("x", Add (Var "x", Num 1))  
let e = App (f, Num 2)
```

(fun x -> x + 1) 2

```
let rec eval_lambda = function  
  | Num n → IntValue n  
  | Add (e1, e2) →  
    (match eval_lambda e1, eval_lambda e2 with  
     | IntValue n1, IntValue n2 → IntValue (n1 + n2)  
     | _ → failwith "Not an integer value")  
  | Fun (x, e) → Closure (x, e)  
  | App (e1, e2) →  
    (match eval_lambda e1 with  
     | Closure (x, e) → eval_lambda (subst x e2 e)  
     | _ → failwith "Not a function")  
  | Var x → failwith "Undefined variable"
```


Lambda Calculus - Call by name



```
type lambda =  
  | Num of int  
  | Add of lambda * lambda  
  
  | Var of string  
  | App of lambda * lambda  
  | Fun of string * lambda  
  
let f = Fun ("x", Add (Var "x", Num 1))  
let e = App (f, Num 2)
```

(fun x -> x + 1) 2

```
type value =  
  | IntValue of int  
  | Closure of string * lambda
```

```
let rec eval_lambda = function  
  | Num n → IntValue n  
  | Add (e1, e2) →  
    (match eval_lambda e1, eval_lambda e2 with  
     | IntValue n1, IntValue n2 → IntValue (n1 + n2)  
     | _ → failwith "Not an integer value")  
  | Fun (x, e) → Closure (x, e)  
  | App (e1, e2) →  
    (match eval_lambda e1 with  
     | Closure (x, e) → eval_lambda (subst x e2 e)  
     | _ → failwith "Not a function")  
  | Var x → failwith "Undefined variable"
```


Lambda Calculus - Call by name



```
type lambda =  
  | Num of int  
  | Add of lambda * lambda  
  
  | Var of string  
  | App of lambda * lambda  
  | Fun of string * lambda  
  
let f = Fun ("x", Add (Var "x", Num 1))  
let e = App (f, Num 2)
```

(fun x -> x + 1) 2

```
type value =  
  | IntValue of int  
  | Closure of string * lambda
```

```
let rec eval_lambda = function  
  | Num n → IntValue n  
  | Add (e1, e2) →  
    (match eval_lambda e1, eval_lambda e2 with  
     | IntValue n1, IntValue n2 → IntValue (n1 + n2)  
     | _ → failwith "Not an integer value")  
  | Fun (x, e) → Closure (x, e)  
  | App (e1, e2) →  
    (match eval_lambda e1 with  
     | Closure (x, e) → eval_lambda (subst x e2 e)  
     | _ → failwith "Not a function")  
  | Var x → failwith "Undefined variable"
```

Lambda Calculus - Call by name



```
type lambda =  
  | Num of int  
  | Add of lambda * lambda  
  
  | Var of string  
  | App of lambda * lambda  
  | Fun of string * lambda  
  
let f = Fun ("x", Add (Var "x", Num 1))  
let e = App (f, Num 2)
```

```
type value =  
  | IntValue of int  
  | Closure of string * lambda
```

```
let rec eval_lambda = function  
  | Num n → IntValue n  
  | Add (e1, e2) →  
    (match eval_lambda e1, eval_lambda e2 with  
     | IntValue n1, IntValue n2 → IntValue (n1 + n2)  
     | _ → failwith "Not an integer value")  
  | Fun (x, e) → Closure (x, e)  
  | App (e1, e2) →  
    (match eval_lambda e1 with  
     | Closure (x, e) → eval_lambda (subst x e2 e)  
     | _ → failwith "Not a function")  
  | Var x → failwith "Undefined variable"
```

Lambda Calculus - Call by name



```
type lambda =  
  | Num of int  
  | Add of lambda * lambda  
  
  | Var of string  
  | App of lambda * lambda  
  | Fun of string * lambda  
  
let f = Fun ("x", Add (Var "x", Num 1))  
let e = App (f, Num 2)
```

```
type value =  
  | IntValue of int  
  | Closure of string * lambda
```

```
let rec eval_lambda = function  
  | Num n → IntValue n  
  | Add (e1, e2) →  
    (match eval_lambda e1, eval_lambda e2 with  
     | IntValue n1, IntValue n2 → IntValue (n1 + n2)  
     | _ → failwith "Not an integer value")  
  | Fun (x, e) → Closure (x, e)  
  | App (e1, e2) →  
    (match eval_lambda e1 with  
     | Closure (x, e) → eval_lambda (subst x e2 e)  
     | _ → failwith "Not a function")  
  | Var x → failwith "Undefined variable"
```


Lambda Calculus - Call by name



```
type lambda =  
  | Num of int  
  | Add of lambda * lambda  
  
  | Var of string  
  | App of lambda * lambda  
  | Fun of string * lambda  
  
let f = Fun ("x", Add (Var "x", Num 1))  
let e = App (f, Num 2)
```

```
type value =  
  | IntValue of int  
  | Closure of string * lambda
```

```
let rec eval_lambda = function  
  | Num n → IntValue n  
  | Add (e1, e2) →  
    (match eval_lambda e1, eval_lambda e2 with  
     | IntValue n1, IntValue n2 → IntValue (n1 + n2)  
     | _ → failwith "Not an integer value")  
  | Fun (x, e) → Closure (x, e)  
  | App (e1, e2) →  
    (match eval_lambda e1 with  
     | Closure (x, e) → eval_lambda (subst x e2 e)  
     | _ → failwith "Not a function")  
  | Var x → failwith "Undefined variable"
```


Lambda Calculus - Call by name

```
eval_lambda App (Fun ("x", Add (Var "x", Num 1)), Num 2)
  eval_lambda Fun ("x", Add (Var "x", Num 1))
    = Closure ("x", Add (Var "x", Num 1))

  eval_lambda subst "x" (Num 2) (Add (Var "x", Num 1))
    = eval_lambda (Add (Num 2, Num 1))
    = IntValue 3

= IntValue 3
```

[55]

... - : value = IntValue 3

Lambda Calculus - Call by name

- **Call-by-name** means that the argument of a function is not evaluated before the function call, but instead, it is expanded within the function body before being evaluated.
- Alternatives are:
 - **Call-by-value**: evaluate before calling the function.
 - **Call-by-need**: evaluate only when used, but only once.

```
let rec eval_lambda = function
| Num n → IntValue n
| Add (e1, e2) →
    (match eval_lambda e1, eval_lambda e2 with
     | IntValue n1, IntValue n2 → IntValue (n1 + n2)
     | _ → failwith "Not an integer value")
| Fun (x, e) → Closure (x, e)
| App (e1, e2) →
    (match eval_lambda e1 with
     | Closure (x, e) → eval_lambda (subst x e2 e)
     | _ → failwith "Not a function")
| Var x → failwith "Undefined variable"
```

Alonzo Church (1903 - 1995)

- Foundational work on Programming Languages.
- Devised the formal system of Lambda Calculus:

$$E ::= x \mid \lambda x.E \mid E E$$

$$(\lambda x.E) E' \longrightarrow E\{E'/x\} \qquad \frac{E \longrightarrow E''}{E E' \longrightarrow E'' E'}$$

The End!

